

Enterprise Portfolio Management System

Project Proposal for UCS503P

Submitted to: Dr. Raghav B. Venkataramaiyer

Thapar Institute of Engineering and Technology

Your Name, CSED | Roll No: YOUR_ROLL_NO | Email: YOUR_EMAIL
Friend's Name, CSED | Roll No: FRIEND_ROLL_NO | Email: FRIEND_EMAIL

Higher-order goal *secondary framing*

This project aims to improve the efficiency, transparency, and organization of portfolio management by providing a centralized platform for portfolio managers and their clients. The system will consolidate portfolio holdings, transactions, market information, performance analysis, risk metrics, and client investment requests into a single web-based application.

While the broader domain is investment and wealth management, the immediate engineering objective is to build a deployable and measurable software system that supports portfolio-management workflows, provides timely portfolio information, and maintains a consistent and auditable representation of client portfolios.

Time-to-value *primary heuristic*

- We will deliver a usable portfolio-management workflow early by first implementing authentication, client management, portfolio creation, holdings, transactions, and basic portfolio valuation.
- Development will follow short iterations, with each iteration producing a functional increment that can be tested independently.
- CI-backed build and testing will be introduced early so that changes made by either team member can be validated continuously.
- Later iterations will progressively introduce market-data integration, portfolio analytics, risk analysis, investment-request workflows, reporting, and AI-assisted explanations.
- Success will be measured by the correctness and responsiveness of the core portfolio workflow in the initial iteration and improved through subsequent iterations.

Problem Statement

Portfolio managers handling multiple clients need to continuously monitor holdings, transactions, portfolio value, market prices, performance, asset allocation, and risk. In a fragmented or manually managed workflow, this can create several problems:

- **Fragmentation:** client information, portfolio holdings, transactions, market information, and reports may be maintained across separate tools.
- **Limited portfolio visibility:** managers may not have a single, consistent view of the current state of each client's portfolio.
- **Manual calculations:** portfolio value, profit/loss, allocation, and performance metrics can require repeated manual calculations.
- **Delayed market information:** decisions based on outdated prices can result in an inaccurate view of current portfolio value.
- **Inefficient client-manager workflow:** investment requests and subsequent decisions may require communication outside the portfolio-management system.
- **Limited risk visibility:** portfolio concentration, volatility, drawdown, and other risk indicators may not be available in a unified dashboard.
- **Reporting overhead:** generating portfolio and performance reports can require manual effort.

- **Auditability:** portfolio changes and managerial actions require a consistent history so that changes can be traced.

The proposed system addresses these issues through a centralized, role-based portfolio-management platform.

Proposed Solution *Software Proposition*

Overview

Build a web-based **Enterprise Portfolio Management System (EPMS)** with separate interfaces for portfolio managers and clients.

- **Client portal:** clients can view their portfolio, holdings, performance, current market prices, transactions, reports, and submit investment requests.
- **Portfolio manager portal:** managers can monitor assigned clients, manage portfolios, review holdings and transactions, analyze performance and risk, and approve or reject investment requests.
- **Portfolio engine:** maintains holdings, transactions, cash balances, portfolio value, cost basis, and profit/loss.
- **Market-data layer:** retrieves market information through a supported market-data provider and propagates relevant price updates to the application.
- **Analytics layer:** calculates portfolio performance, allocation, and selected risk metrics.
- **Reporting layer:** generates portfolio and performance reports.
- **AI assistant:** uses a locally hosted Ollama model to explain portfolio information and analytics using verified data supplied by the backend.
- **Audit and notification layer:** records important portfolio actions and provides notifications for relevant workflow events.

Core workflow

- 1 A client logs into the client portal and views their portfolio.
- 2 The system displays current holdings, portfolio value, P&L, allocation, and performance.
- 3 The client submits an investment request for a selected asset and amount.
- 4 The portfolio manager receives the request in the manager portal.
- 5 The manager reviews the client's portfolio, allocation, and risk information.
- 6 The manager approves or rejects the request.
- 7 For an approved request, the system records a **simulated transaction** and updates the portfolio state.
- 8 The client receives a notification and can view the updated portfolio.
- 9 Portfolio valuation is updated using the latest available market prices.

The system will therefore maintain a traceable workflow from **client request** → **manager decision** → **portfolio update** → **client visibility**.

Operational constraints for an educational portfolio-management setting

- The system will be implemented as an educational portfolio-management platform and will **not execute real-money trades**.
- Market-data availability will depend on the selected provider and its applicable access requirements.
- The portfolio engine, rather than the AI model, will be responsible for authoritative financial calculations.
- Ollama will be used for natural-language explanations and analysis based on application data rather than autonomous trading decisions.

- The implementation will prioritize workflow correctness, security, deployability, and measurable performance rather than research-level financial algorithms.

Solution Approach *Engineering focus*

This is an engineering course project, so the focus is on reliable portfolio-management workflows, correctness, security, scalability, and measurable delivery rather than novel financial-algorithm research.

Portfolio management and financial calculations *non-research*

The backend will maintain clients, portfolio accounts, assets, holdings, transactions, and cash balances.

The portfolio engine will calculate:

- current portfolio value
- cost basis
- realized P&L
- unrealized P&L
- portfolio return
- asset allocation

Financial calculations will be performed by the application backend using appropriate numeric representations rather than delegated to the language model.

Web architecture *web-based solution*

The system will use a layered web architecture:

- **Frontend:** React + TypeScript for client and portfolio-manager interfaces.
- **Backend API:** Java + Spring Boot for authentication, authorization, portfolio management, transactions, investment requests, calculations, analytics, market-data processing, notifications, and AI integration.
- **Database:** PostgreSQL for persistent application data.
- **Caching:** Redis for appropriate caching and real-time data handling.
- **AI:** Ollama for locally hosted natural-language portfolio explanations.
- **Market data:** Market Data Service / DhanHQ for available market information.

The frontend will communicate with the backend through REST APIs and WebSocket connections where appropriate. PostgreSQL will store persistent application data, while Redis will be used for appropriate caching and real-time data handling.

CI/CD and fast delivery enabler

- automatically build the backend and frontend
- run automated tests on changes
- validate pull requests
- produce repeatable deployment artifacts
- support frequent releases of incremental functionality

Both team members will work using Git branches and pull requests rather than directly modifying the stable main branch.

Evaluation Criterion *measurable and attributable*

We define success using measurable metrics tied to the core portfolio workflow.

Primary evaluation metric

Portfolio valuation correctness: for predefined test portfolios containing known holdings, transactions, and prices, the calculated portfolio value and P&L should match independently calculated expected values within a predefined numerical tolerance.

Attribution will be based on stored transactions, recorded quantities, cost basis, market prices, and backend calculation results.

Secondary evaluation metrics

- **Market-data propagation latency:** time between receiving a market-price update and displaying the corresponding updated price on the dashboard.
- **API response time:** response times for common operations such as login, portfolio retrieval, holdings retrieval, transaction retrieval, and investment-request retrieval.
- **Workflow correctness:** investment requests must follow the expected state transitions from PENDING to APPROVED or REJECTED, with approved requests updating the appropriate simulated portfolio records.
- **Access-control correctness:** clients must access only their own portfolios, managers must access only assigned portfolios, and clients must not be able to approve investment requests.
- **System reliability:** successful execution of the core workflow during the project pilot/staging period.

Pilot validation plan *high-level*

We will conduct a controlled demonstration using simulated users and portfolios:

- 2–3 portfolio managers
- 5–10 simulated clients
- multiple portfolios and assets
- predefined transactions
- predefined market-price scenarios

The pilot will test client login, portfolio viewing, investment-request submission, manager review, approval/rejection, portfolio updates, market-price updates, P&L recalculation, access-control rules, and AI-generated portfolio explanations.

Scalability *with foresight*

The system should support growth in the number of clients, portfolios, holdings, transactions, and concurrent dashboard users.

Scalable (theoretically): the system will support growth through:

- separation of frontend and backend
- stateless REST APIs where appropriate
- database indexing
- pagination for large datasets
- Redis caching
- WebSocket-based streaming for market updates

Operationally deployable (practically): the application will be containerized using Docker and designed to run on commonly available hosting infrastructure. The CI/CD pipeline will provide repeatable builds and deployments.

Engine availability heuristic

A mature set of tools and services is available to implement this as a web-based project:

- **Frontend:** React + TypeScript.

- **Backend:** Java + Spring Boot.
- **Database:** PostgreSQL.
- **ORM:** Hibernate / Spring Data JPA.
- **Authentication:** Spring Security + JWT.
- **Cache:** Redis.
- **Market data:** DhanHQ, subject to applicable access requirements.
- **AI:** Ollama with a locally hosted language model.
- **Testing:** JUnit and integration testing.
- **Version control:** Git + GitHub.
- **CI/CD:** GitHub Actions.
- **Deployment:** Docker.

We will use these mature components to focus on workflow design, correctness, measurement, security, and deployability rather than inventing new infrastructure.

Project scope and deliverables *iteration-friendly*

Initial deliverable *first iteration*

The first iteration will provide a usable core portfolio-management workflow:

- role-based authentication
- client and manager profiles
- client-manager assignment
- portfolio creation
- asset and holding management
- transaction recording
- basic portfolio valuation
- basic P&L calculation
- initial manager dashboard
- initial client portfolio view
- backend unit tests
- CI build and test pipeline

Subsequent deliverables

- **Iteration 2 – Market data and performance:** market-data integration, live price updates, portfolio valuation using current prices, P&L updates, performance charts, and asset allocation.
- **Iteration 3 – Client-manager workflow:** client investment requests, manager approval/rejection, simulated transactions, notifications, transaction history, and audit logging.
- **Iteration 4 – Risk and reporting:** volatility, Sharpe ratio, beta, maximum drawdown, concentration analysis, portfolio reports, and benchmark comparison.
- **Iteration 5 – AI and deployment:** Ollama integration, AI portfolio explanations, AI-assisted risk explanations, Dockerized deployment, performance testing, security testing, and CI/CD deployment pipeline.

Risks and mitigations

- **Market-data dependency:** the external market-data provider may be unavailable or subject to access limitations. *Mitigation:* use an abstraction layer for market-data access and maintain simulated/mock market data for development and testing.
- **Financial calculation errors:** incorrect calculations could produce misleading portfolio information. *Mitigation:* implement deterministic backend calculations and validate them using predefined test cases with independently calculated expected results.
- **AI hallucination:** the language model may generate an incorrect explanation or financial claim. *Mitigation:* financial values will be calculated by the backend; Ollama will be used as an explanatory interface rather than an authoritative calculation engine.
- **Unauthorized data access:** a client could potentially access another client's portfolio information. *Mitigation:* implement role-based authorization, ownership checks, protected APIs, and security tests.
- **Team integration conflicts:** parallel development by two team members could cause conflicting changes. *Mitigation:* use feature branches, pull requests, code review, automated tests, and a protected main branch.
- **Evaluation measurement ambiguity:** subjective evaluation may make it difficult to demonstrate whether the system meets its objectives. *Mitigation:* define measurable metrics and reproducible test scenarios before implementation.

Summary

This project proposes a deployable **Enterprise Portfolio Management System** that centralizes portfolio management, market monitoring, performance analysis, risk analysis, client-manager workflows, reporting, and AI-assisted portfolio explanations.

The project follows an iterative engineering approach in which the first iteration delivers a functional portfolio-management workflow and subsequent iterations introduce market-data integration, analytics, investment-request workflows, risk analysis, AI capabilities, and deployment infrastructure.

The system emphasizes measurable correctness, security, responsiveness, scalability, and CI/CD-based development while using mature software components rather than relying on research-level algorithms.

The resulting platform will provide two primary user experiences: a **portfolio manager portal** for managing clients and portfolios and a **client portal** for monitoring investments and interacting with the portfolio-management workflow.